

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Вычислительная математика»
Тема: Решение СЛАУ методом Якоби

Студентка гр. 4342

Волкова В.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Задание

В ходе выполнения работы необходимо реализовать метод Якоби для решения системы линейных алгебраических уравнений с n неизвестными. Программа должна поддерживать генерацию трех типов матриц: случайной, матрицы Гильберта и диагонально-доминантной. Для каждой системы требуется определить, сходится ли итерационный процесс, вычислить относительную невязку решения в L_2 -норме, число обусловленности матрицы и сравнить полученный результат с решением библиотеки SciPy.

Дополнительно необходимо провести вычислительные эксперименты и показать, как структура матрицы влияет на сходимость метода Якоби. В отличие от прямых методов, здесь важно не только существование решения, но и выполнение условий сходимости итерационного процесса.

Теоретические положения

Пусть система линейных уравнений записана в виде $Ax = b$, где A — квадратная матрица коэффициентов, x — вектор неизвестных, b — вектор свободных членов. Метод Якоби относится к итерационным методам. На каждом шаге из предыдущего приближения строится новое, причем все компоненты нового вектора вычисляются независимо друг от друга.

Матрицу A можно представить в виде $A = D + R$, где D — диагональная часть, R — матрица A с нулями на диагонали. Тогда итерационная формула метода Якоби имеет вид $x^{(k+1)} = D^{-1}(b - Rx^{(k)})$. В работе в качестве начального приближения выбирается нулевой вектор.

Критерием остановки служит условие $\|x^{(k+1)} - x^{(k)}\|_{\infty} < \epsilon_{ps}$. Для метода Якоби вопрос сходимости является принципиальным. Достаточным условием сходимости считается строгое диагональное преобладание матрицы. В этом случае итерационная матрица $B = -D^{-1}R$ удовлетворяет условию $\|B\| < 1$, и последовательность приближений сходится к точному решению.

Для оценки качества приближенного решения вычисляется относительная невязка $\delta = \|A\hat{x} - b\|_2 / \|b\|_2$. Кроме того, для сравнения с библиотечным решением используется относительное отличие $\|x_{\text{jacobi}} - x_{\text{scipy}}\|_2 / \|x_{\text{scipy}}\|_2$. Число обусловленности $\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$ показывает чувствительность решения к ошибкам исходных данных и округления.

Выполнение работы

Программа написана на языке Python и поддерживает два режима работы: `solve` — решение одной системы и `research` — проведение серии экспериментов для нескольких размеров и типов матриц. Основные функции программы выполняют генерацию тестовых матриц, реализацию итерационного процесса метода Якоби, вычисление невязки, числа обусловленности и сравнение с ответом SciPy.

`generate_random_system()` генерирует случайную матрицу коэффициентов и случайный вектор правых частей с помощью генератора NumPy.

`generate_hilbert_system()` строит матрицу Гильберта по формуле $a_{ij} = 1 / (i + j - 1)$ и формирует систему с заранее известным точным решением $x = (1, \dots, 1)$.

`generate_diagonal_dominant_system()` создает диагонально-доминантную матрицу, усиливая диагональные элементы так, чтобы их модуль был больше суммы модулей остальных элементов строки.

`jacobi_method()` выполняет итерации метода Якоби, контролирует критерий остановки и обнаруживает явную расходимость.

`iteration_matrix_inf_norm()` вычисляет норму $\|B\|_\infty$ и используется для анализа условий сходимости.

`analyze_system()` объединяет вычисление всех характеристик: числа итераций, относительной невязки, числа обусловленности и отличия от SciPy.

Таблица 1 – Результаты тестирования метода Якоби при $n = 10$

Тип матрицы	Сошелся	Итерации	$\ B\ _{\infty}$	Результат
Случайная	Нет	26	2.081214e+02	Метод не сошелся за заданное число итераций.
Гильберта	Нет	112	1.265666e+01	Метод не сошелся за заданное число итераций.
Диагонально-доминантная	Да	20	9.572924e-01	x1 = -0.0643513844 x2 = -0.2140400441 x3 = -0.1057249484 ... отличие от SciPy = 8.391037e-11

Исследование сходимости и обусловленности

Для диагонально-доминантных матриц метод Якоби должен сходиться, поскольку выполняется достаточное условие сходимости. Для случайных матриц и матриц Гильберта такое условие обычно не выполняется, поэтому метод может расходиться даже тогда, когда система имеет единственное решение.

Таблица 2 – Влияние структуры матрицы на сходимость метода Якоби

n	Тип матрицы	Сошелся	Итерации	$\ B\ _{\infty}$	Невязка	cond(A)	Отличие
10	Случайная	Нет	26	2.081214e+02	—	1.814097e+01	—
10	Гильберта	Нет	112	1.265666e+01	—	1.602441e+13	—
10	Диагонально-доминантная	Да	20	9.572924e-01	8.444074e-11	2.451315e+00	8.391037e-11

п	Тип матрицы	Сошелся	Итерации	$\ B\ _{\infty}$	Невязка	cond(A)	Отличие
50	Случайная	Нет	26	$1.003e+03$	—	$2.828e+02$	—
50	Гильберта	Нет	26	$6.812e+01$	—	$7.960e+18$	—
50	Диагонально-доминантная	Да	13	$9.995e-01$	$6.761e-11$	$1.785e+00$	$6.480e-11$
10000	Диагонально-доминантная	Да	5	$1.000e+00$	$2.036e-10$	$1.048e+00$	$2.036e-10$

Анализ результатов

Эксперименты подтверждают, что для метода Якоби решающую роль играет не только число обусловленности, но и выполнение условий сходимости. Для случайных матриц и матриц Гильберта норма итерационной матрицы $\|B\|_{\infty}$ существенно превышает единицу, поэтому итерационный процесс не стабилизируется и метод не дает надежного ответа в пределах заданного числа шагов.

Для диагонально-доминантных матриц наблюдается противоположная картина: $\|B\|_{\infty}$ оказывается меньше единицы, метод сходится, а найденное решение хорошо согласуется с ответом SciPy. При увеличении размера матрицы число итераций растет, что естественно для итерационного метода, но невязка и отличие от эталонного решения остаются малыми.

Матрица Гильберта дополнительно демонстрирует влияние плохой обусловленности. Даже при существовании точного решения такая система неудобна для метода Якоби, поскольку не удовлетворяет стандартным условиям сходимости. Это показывает важное различие между прямыми и итерационными методами: для последних недостаточно невырожденности

матрицы, необходимо также обеспечить сходимость последовательности приближений.

Выводы

В ходе выполнения лабораторной работы была разработана программа на Python, реализующая метод Якоби для решения систем линейных алгебраических уравнений. Программа поддерживает генерацию случайных, гильбертовых и диагонально-доминантных матриц, вычисляет относительную невязку, число обусловленности и сравнивает полученное решение с результатом SciPy.

Эксперименты показали, что метод Якоби надежно работает для диагонально-доминантных матриц, где выполняется достаточное условие сходимости. Для случайных матриц и матриц Гильберта метод в общем случае не сходится, поэтому использование итерационного подхода без предварительного анализа матрицы может привести к отсутствию результата.

Таким образом, лабораторная работа подтвердила, что для метода Якоби решающую роль играет структура матрицы. При выборе итерационного метода необходимо анализировать как обусловленность задачи, так и саму возможность сходимости итерационного процесса.

Приложение А

Исходный код программы

Файл: jacobi_method.py

```
import argparse
import numpy as np
import scipy.linalg

def parse_args():
    parser = argparse.ArgumentParser(
        description="Решение СЛАУ методом Якоби с исследованием сходимости"
    )

    subparsers = parser.add_subparsers(dest="command", required=True)

    research_parser = subparsers.add_parser("research", help="Провести серию экспериментов")
    research_parser.add_argument(
        "--sizes",
        type=int,
        nargs="+",
        default=[10, 100, 300],
        help="Список размеров матриц для исследования"
    )
    research_parser.add_argument(
        "--type",
        choices=["all", "random", "hilbert", "diagonal-dominant"],
        default="all",
        help="Тип матрицы"
    )
    research_parser.add_argument("--interval", type=float, default=100.0,
        help="Верхняя граница генерации")
    research_parser.add_argument("--seed", type=int, default=1, help="Начальное значение генератора")
    research_parser.add_argument("--tol", type=float, default=1e-10,
        help="Точность остановки")
    research_parser.add_argument("--max-iter", type=int, default=20000,
        help="Максимум итераций")

    solve_parser = subparsers.add_parser("solve", help="Решить одну систему")
    solve_parser.add_argument("--n", type=int, required=True, help="Размер матрицы")
    solve_parser.add_argument(
        "--type",
        choices=["random", "hilbert", "diagonal-dominant"],
        default="diagonal-dominant",
        help="Тип матрицы"
    )
    solve_parser.add_argument("--interval", type=float, default=100.0,
        help="Верхняя граница генерации")
    solve_parser.add_argument("--seed", type=int, default=1, help="Начальное значение генератора")
    solve_parser.add_argument("--tol", type=float, default=1e-10, help="Точность остановки")
```

```

    solve_parser.add_argument("--max-iter", type=int, default=20000,
help="Максимум итераций")

    return parser.parse_args()

def validate_args(args):
    if getattr(args, "n", 2) < 2:
        raise ValueError("Размер матрицы должен быть не меньше 2")
    sizes = getattr(args, "sizes", None)
    if sizes is not None and any(size < 2 for size in sizes):
        raise ValueError("Все размеры матриц должны быть не меньше 2")
    if args.interval <= 0:
        raise ValueError("Интервал генерации должен быть положительным")
    if args.tol <= 0:
        raise ValueError("Точность должна быть положительной")
    if args.max_iter <= 0:
        raise ValueError("Максимум итераций должен быть положительным")

def generate_random_system(n, interval=100.0, seed=1):
    rng = np.random.default_rng(seed)
    a = rng.uniform(-interval, interval, size=(n, n))
    b = rng.uniform(-interval, interval, size=n)
    return a, b

def generate_hilbert_system(n):
    idx = np.arange(1, n + 1)
    a = 1.0 / (idx[:, None] + idx[None, :] - 1)
    x_exact = np.ones(n)
    b = a @ x_exact
    return a, b, x_exact

def generate_diagonal_dominant_system(n, interval=100.0, seed=1):
    rng = np.random.default_rng(seed)
    a = rng.uniform(-interval, interval, size=(n, n))

    for i in range(n):
        row_sum = np.sum(np.abs(a[i])) - abs(a[i, i])
        a[i, i] = row_sum + rng.uniform(1.0, interval)

    b = rng.uniform(-interval, interval, size=n)
    return a, b

def relative_residual(a, x, b):
    numerator = np.linalg.norm(a @ x - b, ord=2)
    denominator = np.linalg.norm(b, ord=2)
    return numerator / denominator if denominator != 0 else numerator

def relative_difference(x1, x2):
    denominator = np.linalg.norm(x2, ord=2)
    if denominator == 0:
        return np.linalg.norm(x1 - x2, ord=2)
    return np.linalg.norm(x1 - x2, ord=2) / denominator

```



```

def iteration_matrix_inf_norm(a):
    d = np.diag(a)
    if np.any(np.abs(d) < 1e-15):
        return float("inf")
    off_diag = a - np.diag(d)
    b_matrix = -((off_diag.T) / d).T
    return np.linalg.norm(b_matrix, ord=np.inf)

def jacobi_method(a, b, tol=1e-10, max_iter=20000):
    a = a.astype(float).copy()
    b = b.astype(float).copy()
    d = np.diag(a)

    if np.any(np.abs(d) < 1e-15):
        raise ValueError("На диагонали есть нулевой элемент, метод Якоби
неприменим")

    r = a - np.diagflat(d)
    x = np.zeros_like(b)
    previous_diff = np.inf

    for iteration in range(1, max_iter + 1):
        x_new = (b - r @ x) / d

        if not np.all(np.isfinite(x_new)):
            return x, iteration, False, float("inf")

        diff = np.linalg.norm(x_new - x, ord=np.inf)
        if diff < tol:
            return x_new, iteration, True, diff

        if iteration > 25 and (diff > previous_diff * 10 or np.linalg.norm(x_new,
ord=np.inf) > 1e100):
            return x_new, iteration, False, diff

        previous_diff = diff
        x = x_new

    return x, max_iter, False, diff

def make_system(matrix_type, n, interval, seed):
    if matrix_type == "random":
        return generate_random_system(n, interval, seed), None
    if matrix_type == "diagonal-dominant":
        return generate_diagonal_dominant_system(n, interval, seed), None
    if matrix_type == "hilbert":
        a, b, x_exact = generate_hilbert_system(n)
        return (a, b), x_exact
    raise ValueError("Неизвестный тип матрицы")

def analyze_system(a, b, tol=1e-10, max_iter=20000):
    q_inf = iteration_matrix_inf_norm(a)
    x_jacobi, iterations, converged, final_diff = jacobi_method(a, b, tol,
max_iter)
    condition_number = np.linalg.cond(a)

```

```

result = {
    "jacobi_solution": x_jacobi,
    "iterations": iterations,
    "converged": converged,
    "final_diff": final_diff,
    "iteration_matrix_norm": q_inf,
    "condition_number": condition_number,
}

if converged:
    x_scipy = scipy.linalg.solve(a, b)
    result["scipy_solution"] = x_scipy
    result["residual"] = relative_residual(a, x_jacobi, b)
    result["difference"] = relative_difference(x_jacobi, x_scipy)
else:
    result["scipy_solution"] = None
    result["residual"] = None
    result["difference"] = None

return result

def print_solution_table(result, n, x_exact=None):
    limit = min(n, 20)
    print(f"{'i':<5}{'Метод Якоби':<24}{'SciPy':<24}")
    print("-" * 70)
    if result["converged"]:
        for i in range(limit):
            print(f"{'i':<5}{result['jacobi_solution'][i]:<24.10f}{result['scipy_solution'][i]:<24.10f}")
            if n > limit:
                print("... показаны только первые 20 компонент ...")
    else:
        print("Метод Якоби не сошелся за заданное число итераций.")
    print("-" * 70)
    print(f"Сходимость: {'да' if result['converged'] else 'нет'}")
    print(f"Число итераций: {result['iterations']}")
    print(f"||B||_inf: {result['iteration_matrix_norm']:.6e}")
    print(f"Число обусловленности: {result['condition_number']:.6e}")
    if result["residual"] is not None:
        print(f"Относительная невязка: {result['residual']:.6e}")
        print(f"Относительное отличие от SciPy: {result['difference']:.6e}")
    if x_exact is not None and result["converged"]:
        max_error = np.max(np.abs(result["jacobi_solution"] - x_exact))
        print(f"Максимальная ошибка относительно точного решения: {max_error:.6e}")

def run_solve(args):
    (a, b), x_exact = make_system(args.type, args.n, args.interval, args.seed)
    result = analyze_system(a, b, args.tol, args.max_iter)

    print(f"Тип матрицы: {args.type}")
    print(f"Размер: {args.n}")
    print_solution_table(result, args.n, x_exact)

def run_research(args):

```

```

matrix_types = [args.type] if args.type != "all" else ["random", "hilbert",
"diagonal-dominant"]

print(
    f'{n:<8}{'Тип':<22}{'Сошелся':<10}{'Итерации':<12}{'||B||_inf':<16}"
    f{'Невязка':<16}{'cond(A)':<16}{'Отличие':<16}"
)
print("-" * 116)

for n in args.sizes:
    for matrix_type in matrix_types:
        (a, b), _ = make_system(matrix_type, n, args.interval, args.seed)
        result = analyze_system(a, b, args.tol, args.max_iter)
        residual_text = f"{result['residual']:.3e}" if result["residual"] is
not None else "-"
        difference_text = f"{result['difference']:.3e}" if
result["difference"] is not None else "-"
        print(
            f"{n:<8}{matrix_type:<22}({'да' if result['converged'] else
'нет'):<10}"

f"{result['iterations']:<12}{result['iteration_matrix_norm']:<16.3e}"

f"{residual_text:<16}{result['condition_number']:<16.3e}{difference_text:<16}"
        )

def main():
    args = parse_args()
    validate_args(args)
    if args.command == "solve":
        run_solve(args)
    else:
        run_research(args)

if __name__ == "__main__":
    main()

```